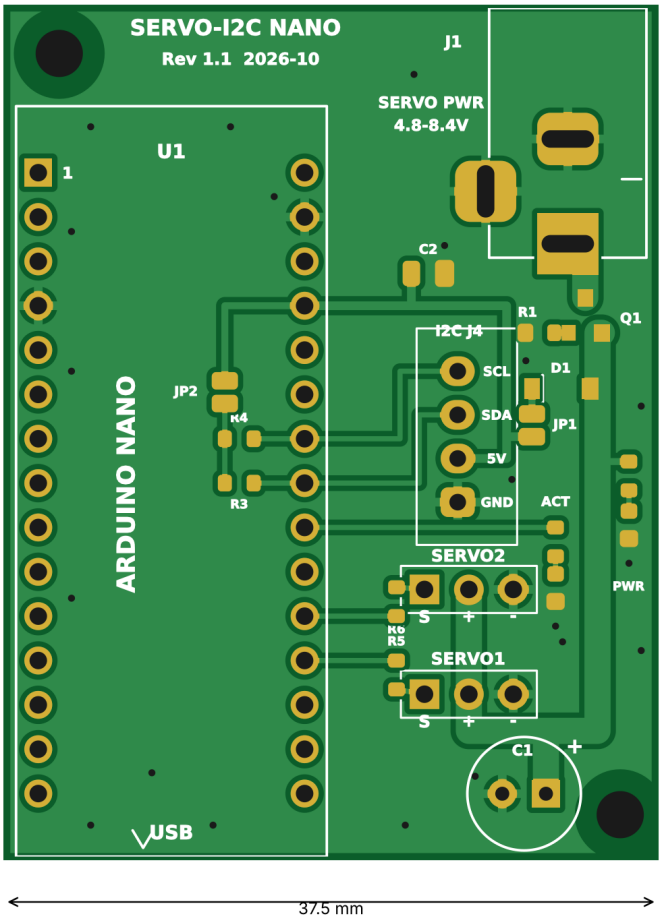
SERVO-I2C NANO

Carte esclave I2C de pilotage de 2 servomoteurs à base d'Arduino Nano

Dossier technique - Révision 1.2 - 29 octobre 2026
Carte rév. 1.1 - maître Arduino UNO R4 WiFi

SERVO-I2C NANO - face dessus (vue de dessus)



Contenu du dossier	
1. Présentation et caractéristiques	6. Nomenclature (BOM)
2. Architecture et choix techniques	7. Faire fabriquer la carte
3. Schéma électrique	8. Assemblage et mise en service
4. Connecteurs et cavaliers	9. Firmware et intégration du protocole
5. Circuit imprimé	10. Maître Arduino UNO R4 WiFi
	11. Points de vigilance

1. Présentation et caractéristiques

La carte SERVO-I2C NANO reçoit des ordres sur un bus I2C (en tant qu'esclave) et pilote deux servomoteurs de modélisme. Elle est construite autour d'un **Arduino Nano** enfiché sur la carte, et les servos sont alimentés par une **source dédiée** (jack DC), totalement indépendante du régulateur de l'Arduino.

Paramètre	Valeur
Dimensions carte	38 x 49 mm, 2 couches, épaisseur 1,6 mm (USB du Nano dépassant de 2,5 mm)
Microcontrôleur	Arduino Nano v3 (ATmega328P, 16 MHz, 5 V) sur barrettes femelles - remplaçable
Bus de commande	I2C esclave 5 V, adresse configurable (0x40 par défaut), 100 / 400 kHz
Connecteur I2C	JST XH 4 points (verrouillable, détrompé) : GND / +5V / SDA / SCL
Sorties servos	2 x barrette 1x3 au pas de 2,54 mm, ordre standard S / + / -
Alimentation servos	Jack DC 5,5 x 2,1 mm, centre +, 4,8 à 8,4 V, 2,5 A max (limite du jack)
Alimentation logique	+5V du bus I2C (par défaut) ou rail servo via diode (option JP1)
Protections	Inversion de polarité (P-MOSFET), résistances série 220 Ω sur les signaux, réservoir 470 µF
Signalisation	LED verte « PWR » : alimentation servos présente ; LED jaune « ACT » (A3) : flash à chaque commande transmise à un servo
Fixation	2 trous M2,5 (Ø 2,7 mm) en diagonale
Dégagement mécanique	≈ 4,2 mm entre le bord du Nano et les fiches servo, ≈ 5 mm jusqu'au connecteur I2C

2. Architecture et choix techniques

Arduino Nano. C'est le plus petit Arduino officiel disposant d'un port USB intégré et de toutes ses broches sur deux rangées au pas standard : il se monte sur barrettes femelles, ce qui le rend remplaçable et reprogrammable en place. Sa logique 5 V évite tout adaptateur de niveau sur un bus I2C 5 V.

Alimentation séparée des servos. Le courant des servos (jusqu'à 1 à 2 A en pointe par servo) ne transite jamais par l'Arduino. Le rail VSERVO part du jack J1, traverse le MOSFET de protection Q1 puis dessert directement les connecteurs J2/J3 par des pistes de 1,5 mm. Les masses sont communes (indispensable pour la référence du signal PWM) et reliées par deux plans de masse cousus par vias.

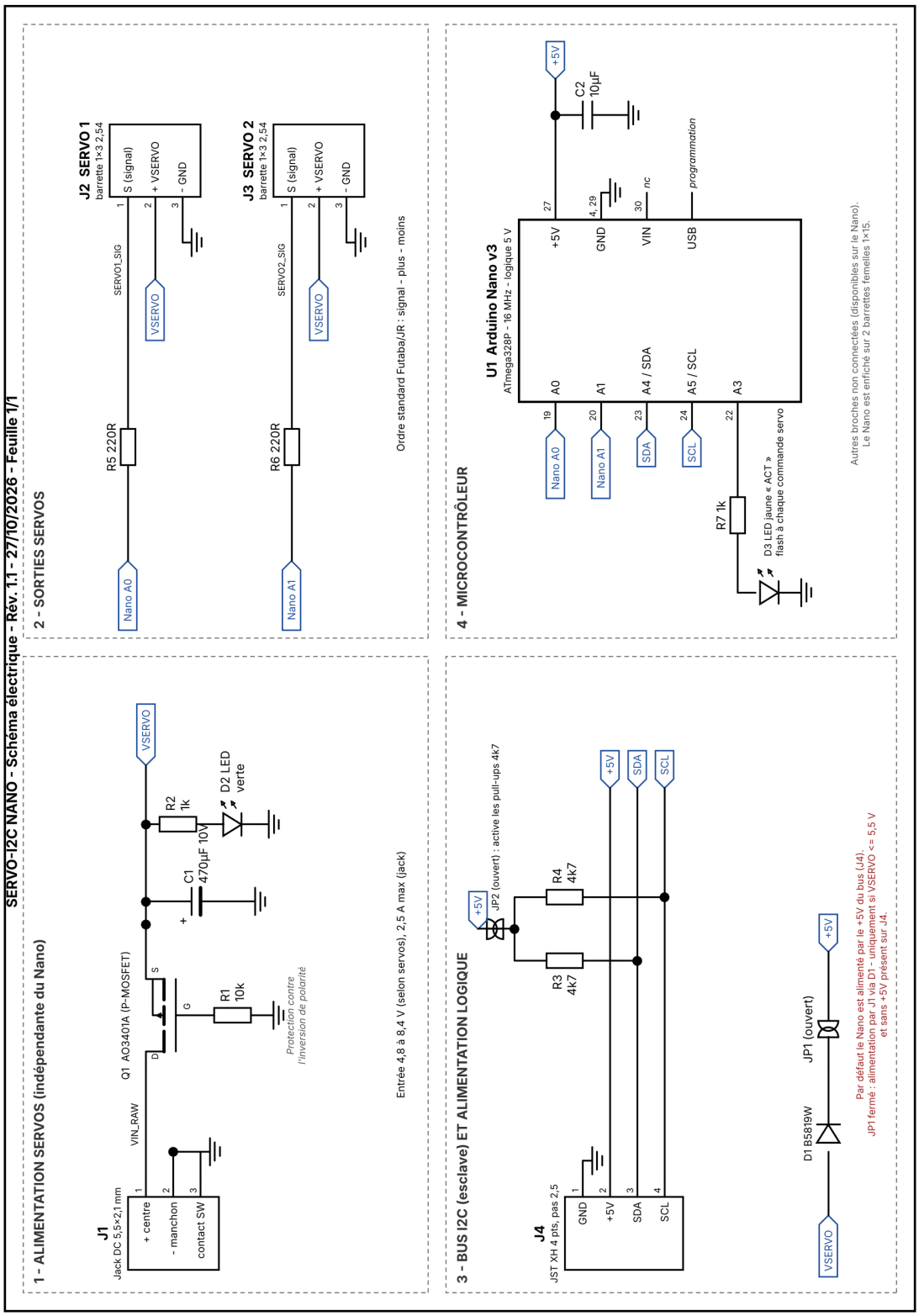
Protection contre l'inversion de polarité. Q1 (AO3401A, P-MOSFET 30 V / 4 A, ~50 mΩ) est monté en « diode idéale » : en polarité correcte il conduit avec une chute négligeable ; en polarité inverse il bloque. C1 (470 µF) absorbe les appels de courant des servos au démarrage.

Alimentation du Nano. Par défaut le Nano est alimenté par la broche +5V du connecteur I2C, fournie par le maître : la logique est ainsi isolée des parasites des servos. Le pont JP1 permet, en option, d'alimenter le Nano depuis J1 à travers la diode Schottky D1 (uniquement si J1 ≤ 5,5 V).

Broches utilisées. A0 et A1 (utilisables en sorties numériques, compatibles avec la bibliothèque Servo) pilotent les servos car elles sont du même côté que les connecteurs, ce qui donne un routage très court. A4/A5 sont les broches I2C matérielles. A3 commande la LED d'activité D3 (jaune, « ACT »).

3. Schéma électrique

Le schéma est également fourni en pleine résolution (schematic.pdf / .svg / .png) dans le dossier docs/.



4. Connecteurs et cavaliers

J1 - Alimentation des servos

Contact	Signal	Remarque
Centre (pin 1)	+ VSERVO (4,8 à 8,4 V)	Tension selon les servos utilisés (6 V typique, 7,4 V pour servos HV)
Manchon (pin 2)	GND	Masse commune avec le Nano et le bus I2C
Contact (pin 3)	GND	Contact de commutation relié à la masse

J2 / J3 - Servos (barrette mâle 1x3, 2,54 mm)

Broche	Signal	Servo 1 (J2)	Servo 2 (J3)
1 (carrée, côté Nano)	S - signal PWM	Nano A0 via R5 220 Ω	Nano A1 via R6 220 Ω
2 (centre)	+ VSERVO	Rail servo	Rail servo
3 (extérieur)	- GND	Masse	Masse

Correspond au câblage standard Futaba/JR/Hitec : fil orange/jaune/blanc = S, rouge = +, marron/noir = -.

J4 - Bus I2C (JST XH 4 points, pas de 2,5 mm)

Broche	Signal	Nano	Remarque
1	GND	GND	Masse de référence
2	+5V	+5V (pin 27)	Alimente le Nano (≈ 20 à 40 mA)
3	SDA	A4	Logique 5 V
4	SCL	A5	Logique 5 V

Cavaliers à souder (ouverts à la livraison)

Repère	Fermé (pont de soudure)	Quand le fermer
JP2	Pull-ups 4,7 kΩ de SDA et SCL vers +5V	Si aucun pull-up n'existe déjà sur le bus. Un seul jeu de pull-ups par bus (ou résistance équivalente ≥ 1,5 kΩ).
JP1	Nano alimenté par J1 via D1	Si le maître ne fournit pas de +5V sur J4 **et** que J1 ≤ 5,5 V. Ne jamais fermer avec un +5V présent sur J4 ni avec J1 > 5,5 V.

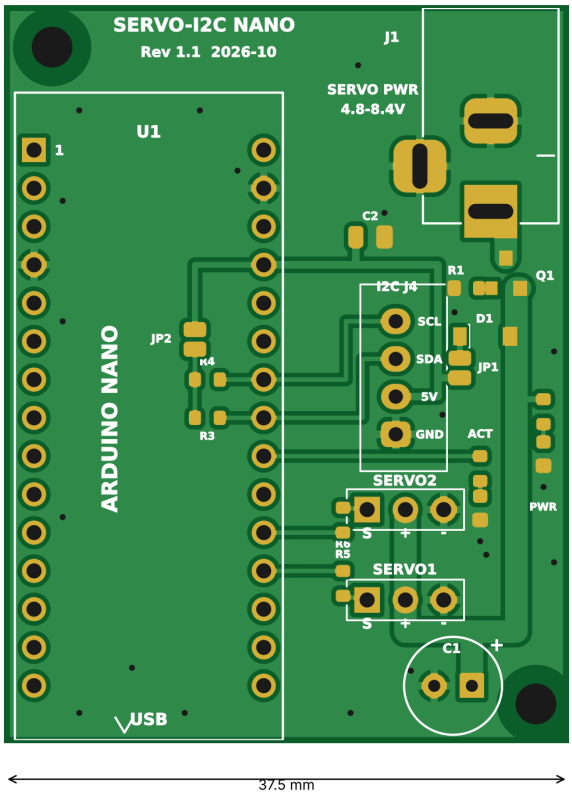
Attention - bus 3,3 V. Le Nano fonctionne en 5 V. Avec un maître 3,3 V (Raspberry Pi, ESP32…), laissez JP2 ouvert, ne reliez pas les pull-ups au 5 V et intercalez un adaptateur de niveau I2C (type PCA9306 ou BSS138) entre le maître et J4 ; le +5V de J4 doit alors être fourni par une alimentation 5 V.

Scénarios d'alimentation

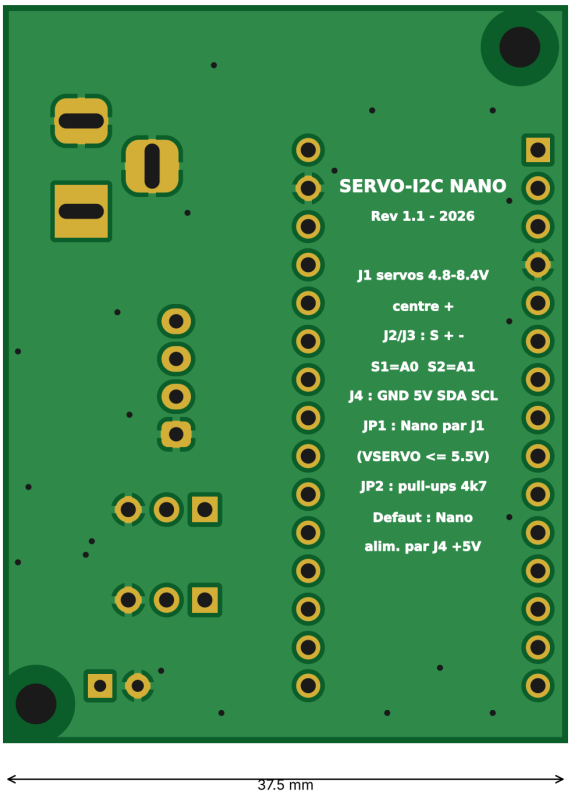
Cas	J4 pin 2	J1	JP1	Résultat
Standard	+5V du maître	Alim servos 4,8-8,4 V	Ouvert	Logique et puissance séparées (recommandé)
Autonome	non connecté	5,0-5,5 V	Fermé	Tout alimenté par J1 (Nano ≈ J1 - 0,3 V)
Programmation	indifférent	indifférent	Ouvert	USB du Nano ; servos actifs seulement si J1 présent

5. Circuit imprimé

SERVO-I2C NANO - face dessus (vue de dessus)



SERVO-I2C NANO - face dessous (vue de dessous)



Rendus générés depuis les fichiers de fabrication. Le dessous porte un aide-mémoire sérigraphié des brochages et cavaliers.

Spécifications de fabrication

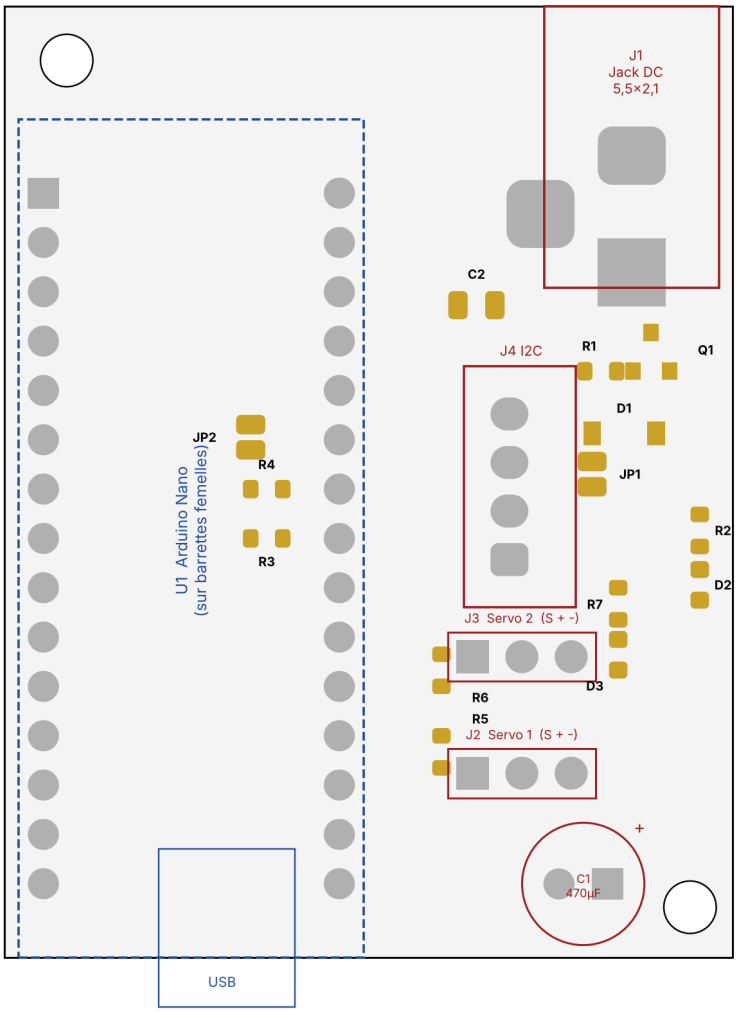
Paramètre	Valeur
Couches	2 (cuivre dessus + dessous, plans de masse sur les deux faces)
Matériau / épaisseur	FR-4 TG130 ou mieux, 1,6 mm
Cuivre	1 oz (35 µm) faces externes
Dimensions	37.5 x 49.0 mm, contour rectangulaire
Piste / isolement min.	0,30 mm / 0,30 mm (plans), 0,20 mm (DRC pistes)
Perçage min. / via	0,40 mm (vias Ø 0,8 mm, couverts de vernis)
Trous oblongs	3 (jack J1), 1,0 x 3,0 mm, métallisés - définis en G85 dans le fichier PTH
Vernis épargne / sérigraphie	Vert / blanc (au choix)
Finition	HASL sans plomb (ENIG possible)
Trous non métallisés	2 x Ø 2,7 mm (fixation M2,5) - fichier NPTH séparé

Fichiers fournis (dossier fabrication/gerber)

Fichier	Contenu
SERVO_I2C_NANO-F_Cu.gtl / B_Cu.gbl	Cuivre dessus / dessous
SERVO_I2C_NANO-F_Mask.gts / B_Mask.gbs	Vernis épargne dessus / dessous
SERVO_I2C_NANO-F_Silkscreen.gto / B_Silkscreen.gbo	Sérigraphie dessus / dessous
SERVO_I2C_NANO-F_Paste.gtp	Crème à braser (pochoir CMS)
SERVO_I2C_NANO-Edge_Cuts.gm1	Contour de la carte
SERVO_I2C_NANO-PTH.drl / NPTH.drl	Perçages métallisés (dont oblongs) / non métallisés (Excellon)
fabrication/assembly/BOM_JLCPCB.csv, CPL_JLCPCB.csv	Nomenclature et placement CMS (format JLCPCB)

Plan d'assemblage

Plan d'assemblage - face composants



6. Nomenclature (BOM)

Repères	Qté	Valeur	Boîtier	Référence	LCSC
C1	1	470uF 10V	CP Radial D6.3mm P2.50mm	Générique 105°C low-ESR	C134793 (à vérifier)
C2	1	10uF 25V X5R	C 0805 2012Metric	Samsung CL21A106KAYNNNE	C15850
D1	1	B5819W	D SOD-123	Changjiang B5819W	C8598
D2	1	LED verte	LED 0603 1608Metric	Hubei KENTO 19-217/GHC	C72043
D3	1	LED jaune	LED 0603 1608Metric	Hubei KENTO 19-217/Y5C	C72038 (à vérifier)
J1	1	Jack DC 5.5x2.1mm	BarrelJack CUI PJ-002A Horizontal	CUI/Same Sky PJ-002A	C2961147 (PJ-002AH, à vérifier)
J2	1	Servo 1	PinHeader 1x03 P2.54mm Vertical	Générique	C35464 (à vérifier)
J3	1	Servo 2	PinHeader 1x03 P2.54mm Vertical	Générique	C35464 (à vérifier)
J4	1	I2C (GND/+5V/SDA/SCL)	JST XH B4B-XH-A 1x04 P2.50mm Vertical	JST B4B-XH-A	C158012 (à vérifier)
JP1,JP2	2	SJ ouvert	SolderJumper-2 P1.3mm Open		
Q1	1	AO3401A	SOT-23	AlphaΩ AO3401A	C15127
R1	1	10k	R 0603 1608Metric	UNI-ROYAL 0603WAF	C25804
R2,R7	2	1k	R 0603 1608Metric	UNI-ROYAL 0603WAF	C21190
R3,R4	2	4k7	R 0603 1608Metric	UNI-ROYAL 0603WAF	C23162
R5,R6	2	220R	R 0603 1608Metric	UNI-ROYAL 0603WAF	C22962
U1	1	Arduino Nano v3 (ATmega328P)	2x PinSocket 1x15 P2.54mm	Arduino A000005 (ou compatible)	
U1-SOCK	2	Barrette femelle 1x15 2,54 mm (x2)	PinSocket 1x15 P2.54mm Vertical	Générique	C2977589 (à vérifier)

Les codes LCSC des CMS correspondent à des références « basic » courantes de JLCPCB. Ceux marqués « à vérifier » (connecteurs, électrolytique) doivent être confirmés dans le catalogue au moment de la commande ; toute référence équivalente de même empreinte convient. Le Nano et ses barrettes ne sont pas assemblés par le fabricant.

Coût indicatif (5 cartes, hors port)

PCB nu : quelques euros les 5 ; assemblage CMS économique : environ 10 à 25 € selon les frais de préparation ; Arduino Nano : 5 € (compatible) à 25 € (officiel) ; connecteurs et condensateur : 2 à 3 € par carte. Ordres de grandeur à vérifier au moment de la commande.

7. Faire fabriquer la carte

Les Gerber sont au format RS-274X (attributs X2), les perçages au format Excellon métrique. Ils sont acceptés par les fabricants usuels (JLCPCB, PCBWay, Eurocircuits, Aisler...). Exemple avec JLCPCB :

Étape	Action
1	Envoyer le fichier **SERVO_I2C_NANO_gerber.zip** (dossier fabrication/) sur la page de devis. Les dimensions 37,5 x 49 mm et 2 couches sont détectées automatiquement.
2	Paramètres : FR-4, 1,6 mm, 1 oz, HASL sans plomb, couleur au choix. Laisser « Remove order number » selon votre préférence (la face arrière est chargée : choisir l'option payante ou accepter le numéro).
3	Vérifier dans la visionneuse les 3 trous oblongs du jack (ils doivent apparaître en oblong).
4	Option assemblage : cocher « PCB Assembly », face Top, puis charger **BOM_JLCPCB.csv** et **CPL_JLCPCB.csv**. Contrôler dans l'aperçu l'orientation de Q1, D1 (barre = cathode, côté JP1), D2 et D3 (cathodes vers le bas).
5	Les composants traversants (J1, J2, J3, J4, C1, barrettes du Nano) se soudent à la main ; ils peuvent aussi être ajoutés en « THT assembly » si le service le propose.

Pour un autre fabricant : envoyer le même ZIP et indiquer « 2 couches, 1,6 mm, 1 oz, slots métallisés présents dans le fichier PTH (G85) ». Les fichiers sont générés en vue de dessus.

8. Assemblage et mise en service

Ordre de soudure

1) CMS (si non assemblés) : R1-R7, C2, D1, D2, D3, Q1. 2) Barrettes femelles 1x15 du Nano (les enficher sur le Nano pendant la soudure pour garantir l'alignement). 3) J2, J3, J4. 4) C1 en respectant la polarité (+ = pastille carrée, côté droit). 5) J1. 6) Ponts JP1/JP2 selon votre configuration.

Contrôles avant la première mise sous tension

#	Test	Attendu
1	Résistance entre centre et manchon de J1 (Nano retiré)	> 1 kΩ (charge de C1 visible)
2	Résistance +5V / GND sur J4	Pas de court-circuit
3	J1 alimenté en 6 V via alim. de labo limitée à 200 mA	LED D2 allumée, 6 V sur la broche 2 de J2/J3
4	J1 alimenté en polarité inverse (test optionnel, courant limité)	LED éteinte, ≈ 0 V sur VSERVO
5	Nano en place, programmé, J4 relié au maître	Le maître détecte l'adresse 0x40 (scan I2C)
6	Envoi d'un ordre de position	Le servo se positionne et la LED jaune « ACT » flashe (60 ms)

9. Firmware et intégration de votre protocole

Le dossier firmware/servo_i2c_nano est un sketch Arduino prêt à compiler (IDE Arduino ou arduino-cli, carte « Arduino Nano », processeur ATmega328P - ou « Old Bootloader » pour beaucoup de clones). Il ne dépend que des bibliothèques standard **Wire** et **Servo**.

Fichier	Rôle
servo_i2c_nano.ino	Cœur : I2C esclave, tampons, gestion des servos, sécurité. Rien à modifier en principe.
config.h	Adresse I2C, plage d'impulsions, durée du flash LED, timeout de sécurité, traces série.
protocole.h / .cpp	**Votre protocole** : décodage des trames reçues et préparation des réponses. Fourni avec un protocole d'exemple à remplacer.
test_maitre/test_maitre.ino	Programme de test pour un Arduino maître utilisant le protocole d'exemple.
maitre_uno_r4_wifi/	Maître WiFi UNO R4 : page web + WebSocket vers I2C (voir section 10).

Principe

Les callbacks I2C s'exécutent sous interruption : ils se contentent de copier la trame reçue. Le décodage est fait dans loop() par protocoleTraiterTrame(), ce qui permet un traitement aussi riche que nécessaire sans bloquer le bus. Les réponses au maître sont préparées à l'avance via reponseDefinir() et envoyées lorsque le maître lit.

Ce que vous avez à écrire

```
bool protocoleTraiterTrame(const uint8_t* trame, uint8_t len) {
  // 1. valider la trame (longueur, commande, CRC éventuel...)
  // 2. appeler servoEcrireAngle(idx, deg) ou servoEcrireMicros(idx, us)
  // 3. préparer la réponse : reponseDefinir(buffer, n)
  return true; // trame valide
}
```

Service fourni	Effet
servoEcrireAngle(idx, deg)	Position 0-180° (idx 0 = servo 1 / J2, 1 = servo 2 / J3)
servoEcrireMicros(idx, us)	Impulsion en µs, bornée par SERVO_US_MIN/MAX (500-2500 par défaut)
servoRelacher(idx)	Arrête les impulsions : le servo n'est plus asservi
servoLireMicros(idx)	Dernière consigne (0 si relâché)
reponseDefinir(data, len)	Données renvoyées à la prochaine lecture du maître (32 octets max)
trameAEtePerdue()	Vrai si une trame est arrivée avant la fin du traitement de la précédente

Contraintes à connaître

• Trames limitées à **32 octets** par transaction (tampon Wire de l'AVR). • La bibliothèque Servo utilise le Timer1 : les PWM matérielles des broches D9/D10 ne sont plus disponibles (sans impact ici). • Les servos ne sont pas activés au démarrage : aucune impulsion n'est émise avant le premier ordre, ce qui évite les à-coups à la mise sous tension. • TIMEOUT_SECURITE_MS permet de relâcher les servos si le maître se tait (perte de liaison). • **LED d'activité** : chaque appel à servoEcrireAngle() ou servoEcrireMicros() fait flasher la LED « ACT » (et la LED L du Nano) pendant LED_ACT_MS, sans bloquer le programme. Une trame invalide ou sans ordre servo ne l'allume pas.

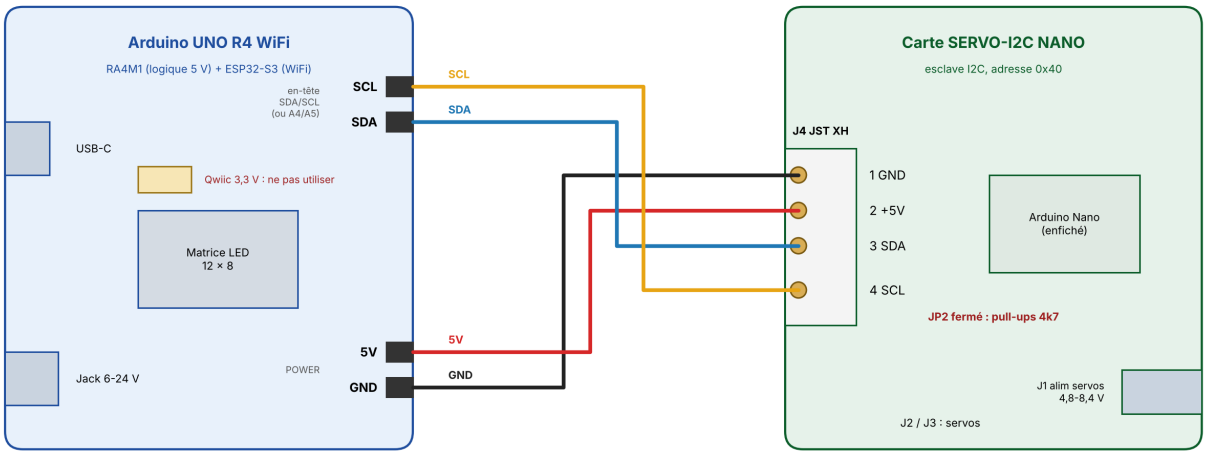
Protocole d'exemple fourni

Trame écrite par le maître	Action
01 ss aa	Servo ss (0/1) à l'angle aa (0-180)
02 ss LL HH	Servo ss à l'impulsion HHLL µs
03 ss	Relâcher le servo ss
10, puis lecture 6 octets	Réponse A5, version, consigne servo 1 (µs, LE), consigne servo 2 (µs, LE)

10. Maître Arduino UNO R4 WiFi

L'Arduino UNO R4 WiFi remplace l'ensemble **Arduino UNO + shield PHPOC** côté maître. Le module ESP32-S3 intégré apporte le WiFi : plus besoin de shield réseau. Le microcontrôleur principal (RA4M1) fonctionne en **logique 5 V**, comme le Nano de la carte : la liaison I2C se fait en direct, sans adaptateur de niveau. **La carte SERVO-I2C NANO est inchangée.**

Câblage du maître Arduino UNO R4 WiFi vers la carte SERVO-I2C NANO



Fil de liaison : câble JST XH 4 points vers fils Dupont mâles, 30 cm max à 100 kHz.
Le 5V du R4 alimente le Nano (≈ 30 mA). Les servos restent alimentés par J1 (alimentation séparée).
Option une seule alimentation 7-8,4 V : la brancher à la fois sur le jack du R4 et sur J1 de la carte.

Câblage

UNO R4 WiFi	Fil	Carte J4	Remarque
GND (POWER)	noir	1 GND	Masse commune obligatoire
5V (POWER)	rouge	2 +5V	Alimente le Nano de la carte (≈ 30 mA)
SDA (en-tête près d'AREF, ou A4)	bleu	3 SDA	Bus Wire, 5 V
SCL (en-tête près d'AREF, ou A5)	jaune	4 SCL	Bus Wire, 5 V

Ne pas utiliser le connecteur Qwiic du R4 : c'est un second bus (Wire1) en 3,3 V. Fermer **JP2** sur la carte pour disposer des pull-ups 4,7 kΩ, sauf si d'autres pull-ups sont déjà présents sur le bus. JP1 reste ouvert : le Nano est alimenté par le 5V du R4.

Alimentation : le R4 est alimenté par USB-C ou par son jack (6 à 24 V). Les servos restent alimentés par J1 de la carte. Avec une seule alimentation de 7 à 8,4 V, on peut la brancher à la fois sur le jack du R4 et sur J1.

Firmware maître (firmware/maitre_uno_r4_wifi)

Fichier	Rôle
maitre_uno_r4_wifi.ino	Connexion WiFi, page de commande sur le port 80, serveur WebSocket sur le port 81, traduction des commandes en trames I2C, reconnexion automatique, matrice de LED.
arduino_secrets.h	Nom et mot de passe du réseau WiFi (2,4 GHz).
config.h	IP fixe optionnelle, ports, sous-protocole WebSocket, adresse I2C de la carte (0x40), fréquence I2C.
carte_servo.cpp / .h	**Construction des trames I2C** : à adapter à votre protocole (même rôle que protocole.cpp côté carte).
page_web.h	Page de commande embarquée : deux curseurs, boutons Centre et Relâcher, lecture d'état.

Installation : dans l'IDE Arduino, installer le paquet de cartes **Arduino UNO R4 Boards** (fournit WiFiS3, Wire et la matrice de LED) et la bibliothèque **WebSockets** de Markus Sattler (Links2004), version 2.5.1 ou plus récente, qui prend en charge l'UNO R4 WiFi. Renseigner arduino_secrets.h,

téléverser, puis lire l'adresse IP dans le moniteur série (115 200 bauds).

Commandes WebSocket

Message texte	Effet	Réponse
S1=90 / S2=45	Servo 1 ou 2 à l'angle donné (0 à 180°)	OK S1=90 ou ERR ...
U1=1500 / U2=1200	Impulsion en µs (500 à 2500)	OK U1=1500 ou ERR ...
R1 / R2	Relâche le servo	OK R1
?	Lit l'état de la carte	ETAT fw=1.0 us1=1500 us2=0

Chaque commande acceptée fait flasher la LED « ACT » de la carte et un cadre sur la matrice du R4. En cas d'erreur I2C, la réponse indique la cause (carte absente, donnée refusée, délai dépassé).

Migration depuis le shield PHPOC

UNO + PHPOC	UNO R4 WiFi
#include <Phpoc.h> / Phpoc.begin()	#include <WiFiS3.h> / WiFi.begin(ssid, mot de passe)
PhpocServer server(80) + server.beginWebSocket("remote_push")	WebSocketsServer sur le port 81 (bibliothèque WebSockets)
Page web stockée dans le shield (PHPoC Debugger)	Page embarquée dans le sketch (page_web.h), servie sur le port 80
ws://ip/remote_push, sous-protocole text.phpoc	ws://ip:81/ ; le sous-protocole text.phpoc est accepté
Configuration réseau dans le shield	arduino_secrets.h et config.h

Pour réutiliser une page web écrite pour le PHPOC, il suffit en général de remplacer l'adresse ws://ip/remote_push par ws://ip:81/ ; le serveur répond avec le sous-protocole « text.phpoc » quand la page le demande. Les ordres reçus sont traités par traiterCommande() : adapter cette fonction si votre page envoie un autre format de message.

11. Points de vigilance

• **Courant** : le jack PJ-002A est donné pour 2,5 A. Pour deux servos puissants (calage > 1,5 A chacun), utiliser une alimentation adaptée et envisager un bornier à vis en révision 2. • **Tension** : C1 est en 10 V ; ne pas dépasser 8,4 V sur J1. • **Longueur du bus I2C** : prévu pour quelques dizaines de centimètres ; au-delà, abaisser la fréquence à 100 kHz voire utiliser un prolongateur de bus (P82B715...). • **Hauteur** : le Nano sur barrettes femelles de 8,5 mm laisse la place aux CMS placés dessous (R3, R4, JP2) ; ne pas souder le Nano à plat. • **Empreinte du jack** : conforme au plan recommandé du PJ-002A (oblongs 1 x 3 mm) ; si vous choisissez un autre modèle, vérifiez sa compatibilité avec ce plan avant commande. • **Validation** : le DRC (isolements, connectivité) a été vérifié par script et les Gerber relus avec un lecteur indépendant ; il est recommandé de commander d'abord une petite série de prototypes.

Arborescence du dossier

```
SERVO_I2C_NANO/
  README.txt
  docs/ (ce PDF, schéma PDF/SVG/PNG, rendus)
  fabrication/SERVO_I2C_NANO_gerber.zip (à envoyer au fabricant)
  fabrication/gerber/ (Gerber + perçages, détail)
  fabrication/assembly/ (BOM, CPL)
  firmware/ (carte, maître UNO R4 WiFi, test maître)
  sources/ (scripts Python ayant généré la carte - modifiables et régénérables)
```

Historique

Rév.	Date	Modification
1.0	27/10/2026	Version initiale (Arduino Nano)
1.1	27/10/2026	Connecteurs décalés de 3,5 mm (dégagement Nano), carte 37,5 x 49 mm ; ajout LED d'activité D3/R7 sur A3
1.2	29/10/2026	Maître Arduino UNO R4 WiFi (remplace UNO + PHPOC) : firmware WiFi/WebSocket, câblage, migration. Carte inchangée (rév. 1.1)